

SVM is normally supervised because it learns from labeled data. One-Class SVM is a special version that can learn what normal data looks like and detect unusual data without requiring normal-vs-abnormal class labels

- SVM tries to draw the best line between the two groups.
- 👉 Decision Boundary
- "Where should I draw the line so that the two groups are separated as well as possible?"

---

Types of SVM: Linear & Non Linear

**SVM is not just "draw a line."**

It does three important things:

Finds a decision boundary Maximizes the margin

Advantages

- 1.High-Dimensional Performance: SVM excels in high-dimensional spaces, making it
- 2.suitable for image classification and gene expression analysis.
- 3.Nonlinear Capability: Utilizing kernel functions like RBF and polynomial SVM effectively handles nonlinear relationships.
- 4.outlier Resilience: SVM can handle outliers by allowing some misclassifications using a soft margin.
- 5.Binary and Multiclass Support: SVM is effective for both binary classification and multiclass classification suitable for applications in text classification.
- 6.Memory Efficiency: It focuses on support vectors making it memory efficient compared to other algorithms

**How its working?**

**Features → Decision Formula → Decision Value → Class → Margin → Correct/Incorrect.**

● Decision Boundary

The line separating the classes.

● Margin

The safe distance around the decision boundary.

● Support Vectors

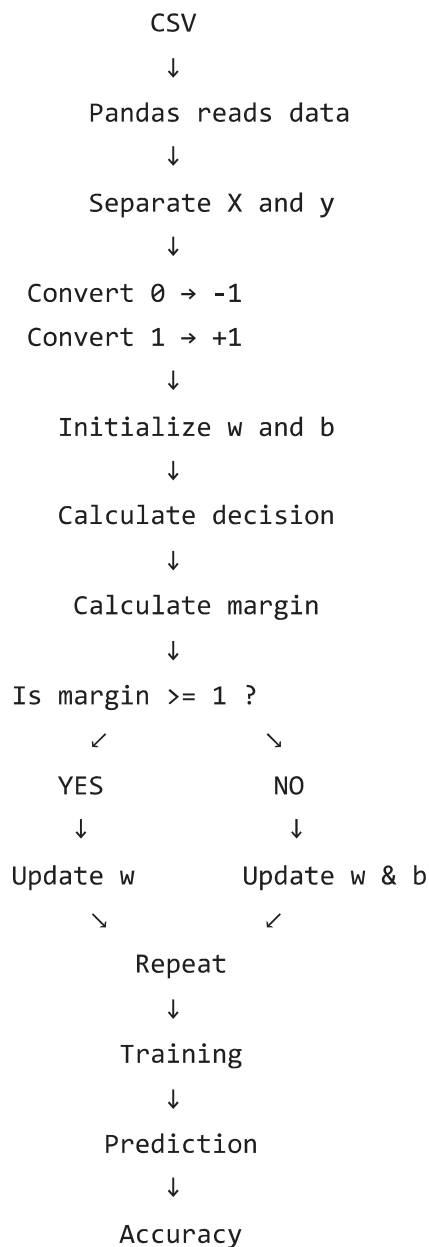
The data points closest to the boundary.



SVM

Finds a boundary with a maximum margin.

# This is formatted as code



Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

1. Decision =  $w \cdot x + b$

2. Decision  $> 0 \rightarrow +1$  Decision  $< 0 \rightarrow -1$
3. Margin  $= y(w \cdot x + b)$
4. Margin  $\geq 1 \rightarrow$  correctly classified
5. Decision Boundary:  $w \cdot x + b = 0$
6. Margin Boundaries:  $w \cdot x + b = +1$   $w \cdot x + b = -1$
7. Closest points  $\rightarrow$  Support Vectors

Decision Formula Decision  $= w_1x_1 + w_2x_2 + b$

$$\begin{aligned}
 &= 2 \times 6 + 0.5 \times 28 - 20 \\
 &= 12 + 14 - 20 \\
 &= 6
 \end{aligned}$$

Since  $6 > 0$ :

$\rightarrow$  Premium (+1)

Then:

Margin  $= y \times$  Decision

$$\begin{aligned}
 &= (+1) \times 6 \\
 &= 6
 \end{aligned}$$

Since  $6 \geq 1$ :

$\rightarrow$  Correctly classified

```

# =====
# SUPPORT VECTOR MACHINE (SVM) - SIMPLE MATHAMETICAL CALCULATION
# =====

# REAL-WORLD EXAMPLE:
# We want to classify a mobile phone as:
#
# -1 = Budget Phone
# +1 = Premium Phone
#
# Features:
# x1 = RAM (GB)
# x2 = Price (₹ Thousand)

```

```

# =====
# STEP 1: GIVEN DATA
# =====

RAM = 6
Price = 28
# SVM learned weights
w1 = 2
w2 = 0.5
# Bias
b = -20
print("===== PHONE DETAILS =====")
print("RAM   =", RAM, "GB")
print("Price =", Price, "₹ Thousand")

# =====
# STEP 2: SVM DECISION FORMULA
# =====
# Formula:
#
# Decision = w1*x1 + w2*x2 + b
print("\n===== SVM FORMULA =====")
print("Formula:")
print("Decision = w1*x1 + w2*x2 + b")

# =====
# STEP 3: SUBSTITUTE THE VALUES
# =====
print("\n===== SUBSTITUTE VALUES =====")
print(
    "Decision =",
    w1, "*", RAM,
    "+",
    w2, "*", Price,
    "+",
    b
)
# =====
# STEP 4: CALCULATE
# =====

decision = (w1 * RAM) + (w2 * Price) + b

print("\n===== CALCULATION =====")

print(
    "Decision =",
    w1 * RAM,
    "+",

```

```

        w2 * Price,
        "+",
        b
    )

print("Decision =", decision)

# =====
# STEP 5: MAKE PREDICTION
# =====

print("\n===== PREDICTION =====")

if decision > 0:
    #y=1
    print("Decision > 0")
    print("Prediction = +1")
    print("Phone Type = PREMIUM")

else:
    #y=0
    print("Decision < 0")
    print("Prediction = -1")
    print("Phone Type = BUDGET")

# =====
# STEP 6: SVM MARGIN
# =====

# Formula:
# 0 or 1
# Margin = y * (w1*x1 + w2*x2 + b)
# y is op of prediction
# Premium = +1
# Budget = -1

y = 1      # Actual class: Premium
margin = y * decision

print("\n===== MARGIN CALCULATION =====")

print("Formula:")
print("Margin = y * (w1*x1 + w2*x2 + b)")
print("\nSubstitution:")
print(
    "Margin =",
    y,
    "*",

```

```

        "(" ,
        w1, "*", RAM,
        "+",
        w2, "*", Price,
        "+",
        b,
        ")"
    )
print("\nMargin =", margin)
# =====
# STEP 7: CHECK THE MARGIN
# =====

print("\n===== MARGIN CHECK =====")

if margin >= 1:
    print("Margin >= 1")
    print("Correctly classified!")
    print("Point is outside the margin.")

else:
    print("Margin < 1")
    print("Point is inside the margin or incorrectly classified.")
# =====
# STEP 8: DECISION BOUNDARY
# =====

# Decision Boundary Formula:
#
#  $w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$ 

print("\n===== DECISION BOUNDARY =====")

print("Formula:")
print(" $w_1 \cdot x_1 + w_2 \cdot x_2 + b = 0$ ")

print("\nOur Boundary:")

print(
    w1,
    "* RAM +",
    w2,
    "* Price +",
    b,
    "= 0"
)

print("\nTherefore:")

print("2 * RAM + 0.5 * Price - 20 = 0")

```

```
# =====
# FINAL SUMMARY
# =====

print("\n=====")
print("          SVM SUMMARY")
print("=====")

print("RAM          :", RAM, "GB")
print("Price         :", Price, "₹ Thousand")
print("Decision Value :", decision)
print("Prediction      :", "Premium" if decision > 0 else "Budget")
print("Margin          :", margin)
print("Correctly Classified :", "YES" if margin >= 1 else "NO")

print("\nKey Formula:")
print("Decision = w1*x1 + w2*x2 + b")

print("\nMargin Formula:")
print("Margin = y * (w1*x1 + w2*x2 + b)")

print("\nDecision Boundary:")
print("w1*x1 + w2*x2 + b = 0")
```

```
import pandas as pd
import numpy as np
# -----
# STEP 1: Read CSV
# -----
df = pd.read_csv("mobile_phones_1000.csv")

print("Dataset:",df)
# -----
# STEP 2: Take input and output
# -----
X = df[["ram_gb", "price_thousand"]].values
y = df["target"].values
# Convert:
# 0 → -1
# 1 → +1
y = np.where(y == 0, -1, 1)
print("\nX values:",X)
print("\ny values:",y)
# -----
# STEP 3: SVM parameters manual
# -----
w1 = -4
w2 = 1
b = -5
```

```

print("\nSVM Formula:")
print("Decision = w1 × RAM + w2 × Price + b")
print("\nSVM Parameters:")
print("w1 =", w1)
print("w2 =", w2)
print("b =", b)

# -----
# STEP 4: Calculate Decision
# -----

print("\nCalculations:\n")

predictions = []

for i in range(len(X)):

    ram = X[i][0]
    price = X[i][1]

    decision = w1 * ram + w2 * price + b

    # Decision > 0 → Premium
    # Decision < 0 → Budget

    if decision >= 0:
        prediction = 1
        class_name = "Premium"
    else:
        prediction = -1
        class_name = "Budget"

    predictions.append(prediction)

    print(
        "RAM =", ram,
        "| Price =", price,
        "| Decision =", decision,
        "| Prediction =", class_name)

# -----
# STEP 5: Accuracy
# -----

predictions = np.array(predictions)
accuracy = np.mean(predictions == y)
print("\nAccuracy =", accuracy * 100, "%")

```

Start coding or [generate](#) with AI.

## Final Customer Groups

```
# This is formatted as code
Customer Data
    ↓
Choose K = 3
    ↓
Randomly select 3 centroids
    ↓
Calculate distance
    ↓
Assign each customer to nearest centroid
    ↓
Calculate new centroids
    ↓
Repeat
    ↓
Final Customer Groups
```

**\*\*The distance formula\*\***

For a customer:

Customer = (Age, Income)

Centroid = (Age, Income)

We calculate:

Distance =  
 $\sqrt{(Age_1 - Age_2)^2 + (Income_1 - Income_2)^2}$

In Python:

```
np.sqrt(np.sum((X[i] - centroids[j]) ** 2))
```

```
#k means clustering unsupervised learning
import pandas as pd
import numpy as np

# Customer dataset
data = pd.DataFrame({
    "Age": [20, 22, 25, 27, 45, 48, 50, 52, 30, 32],
    "Income": [20, 22, 25, 28, 60, 65, 70, 72, 40, 42]
})
```

```
print("Customer Data:")
print(data)

# Convert data into NumPy array
X = data.values.astype(float)

# Number of clusters
K = 3

# Initial centroids
centroids = np.array([
    [22, 22],
    [48, 62],
    [31, 41]
], dtype=float)

# Repeat clustering
for iteration in range(5):

    # Calculate distance from every customer to every centroid
    distances = np.zeros((len(X), K))

    for i in range(len(X)):
        for j in range(K):
            distances[i][j] = np.sqrt(
                np.sum((X[i] - centroids[j]) ** 2)
            )

    # Assign each customer to nearest centroid
    clusters = np.argmin(distances, axis=1)

    # Calculate new centroids
    for j in range(K):
        points = X[clusters == j]

        if len(points) > 0:
            centroids[j] = np.mean(points, axis=0)

    print("\nIteration:", iteration + 1)
    print("Centroids:")
    print(centroids)

# Add cluster to dataset
data["Cluster"] = clusters

print("\nFinal Customer Groups:")
print(data)
```

```
Customer Data:
Age  Income
```

|   |    |    |
|---|----|----|
| 0 | 20 | 20 |
| 1 | 22 | 22 |
| 2 | 25 | 25 |
| 3 | 27 | 28 |
| 4 | 45 | 60 |
| 5 | 48 | 65 |
| 6 | 50 | 70 |
| 7 | 52 | 72 |
| 8 | 30 | 40 |
| 9 | 32 | 42 |

Iteration: 1

Centroids:

|        |        |
|--------|--------|
| [[23.5 | 23.75] |
| [48.75 | 66.75] |
| [31.   | 41. ]] |

Iteration: 2

Centroids:

|        |        |
|--------|--------|
| [[23.5 | 23.75] |
| [48.75 | 66.75] |
| [31.   | 41. ]] |

Iteration: 3

Centroids:

|        |        |
|--------|--------|
| [[23.5 | 23.75] |
| [48.75 | 66.75] |
| [31.   | 41. ]] |

Iteration: 4

Centroids:

|        |        |
|--------|--------|
| [[23.5 | 23.75] |
| [48.75 | 66.75] |
| [31.   | 41. ]] |

Iteration: 5

Centroids:

|        |        |
|--------|--------|
| [[23.5 | 23.75] |
| [48.75 | 66.75] |
| [31.   | 41. ]] |

Final Customer Groups:

|   | Age | Income | Cluster |
|---|-----|--------|---------|
| 0 | 20  | 20     | 0       |
| 1 | 22  | 22     | 0       |
| 2 | 25  | 25     | 0       |
| 3 | 27  | 28     | 0       |
| 4 | 45  | 60     | 1       |
| 5 | 48  | 65     | 1       |
| 6 | 50  | 70     | 1       |
| 7 | 52  | 72     | 1       |
| 8 | 30  | 40     | 2       |
| 9 | 32  | 42     | 2       |

```
import matplotlib.pyplot as plt
import seaborn as sns

# Visualize the clusters
plt.figure(figsize=(10, 7))

# Plot data points, colored by cluster
sns.scatterplot(x='Age', y='Income', hue='Cluster', data=data, palette='viridis')

# Plot centroids
plt.scatter(centroids[:, 0], centroids[:, 1], s=300, c='red', marker='X', label='Centroids')

plt.title('K-Means Clustering of Customer Data (Age vs. Income)')
plt.xlabel('Age')
plt.ylabel('Income')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.show()
```

